

Monte Carlo Simulation - Project

Emil Beck Aagaard, Kamille Møller Frost

October 22, 2025

1 Portfolio loss and importance sampling.

Let $V(S)$ be the value of a portfolio value consisting of financial instruments based on m market prices $S \in \mathbb{R}^m$. The goal of this project is to use Monte Carlo to estimate the portfolio loss L over short time periods ΔT . Here, it is common to use the delta-gamma representation, where the loss is written as

$$L := \delta^\top \Delta S + (\Delta S)^\top \Gamma \Delta S,$$

where $\delta \in \mathbb{R}^m$, $\Gamma \in \mathbb{R}^{m \times m}$ and $\Delta S \sim N(0, \Sigma_S)$ for some covariance matrix Σ_S . Sometimes, it is helpful to consider the linear approximation

$$Q := \delta^\top \Delta S.$$

First, assume that $m = 1, \delta = 2, \Gamma = \Sigma_S = 1$, and $x = 10$. Please always justify your answer.

```
[344]: # setup (only needed once, this cell is hidden in export)
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from scipy.stats import norm, multivariate_normal, ncx2
```

1.1 Questions

Exercise a) Write a program to estimate the exceedance probabilities $\mathbb{P}(L > x)$ by crude Monte Carlo simulation.

For a better comparison of our results, we first derive the analytical probability, for the simple case $m = 1, \delta = 2$ and $\Gamma = \Sigma_S = 1$. Using these parameters we can write:

$$\mathbb{P}(L > x) = \mathbb{P}(2Z + Z^2 > x) \quad Z \sim N(0, 1)$$

Completing the square we get $2Z + Z^2 = (Z + 1)^2 - 1 = Y - 1$ where $Y \sim X_{nc}^2(\nu = 1, \lambda = 1)$, and where $X_{nc}^2(\nu, \lambda)$ denotes a non-central chi-square distribution with ν degrees of freedom and non-centrality parameter λ . In particular the analytical probability is can be calculated by:

$$\mathbb{P}(L > x) = 1 - \mathbb{P}(Y^2 - 1 \leq x) = 1 - \mathbb{P}(Y^2 \leq x + 1)$$

This is determined to be

```
[345]: x = 10.0
analytical_prop = 1-ncx2.cdf(x+1, df=1, nc=1)
print(f"Analytical probability P(L > {x}): {analytical_prop:.6f}")
```

Analytical probability P(L > 10.0): 0.010270

We now implement a crude Monte Carlo simulation, of this probability. We first define:

```
[346]: np.random.seed(2)  # for reproducibility
# assumptions
m = 1
delta = 2.0
gamma = 1.0
sigma = np.array([[1.0]]) # covariance matrix

def loss(m, delta, delta_S, gamma):
    """ Compute the loss L given delta, gamma, sigma and delta_S, note that all
    ↪ parameters should be of dimension m."""
    if m == 1:
        Q = delta*delta_S
        L = Q + (delta_S * gamma) * delta_S
    else:
        Q = delta_S@delta
        L = Q + np.sum((delta_S @ gamma) * delta_S, axis=1)
    return L
```

Crude Monte Carlo is then done by finding N realizations of $\Delta S \sim N(0,1)$. For each realization we then determine the *empirical* loss $\tilde{L}$ and weather or not $\tilde{L} < x$. In particular the Law of large numbers then gives that

$$\frac{\mathbf{1}_{L_1 < x} + \dots + \mathbf{1}_{L_N < x}}{N} \xrightarrow{N \rightarrow \infty} \mathbb{E}[\mathbf{1}_{L_1 < x}] = \mathbb{P}(L_1 < x) = 0.010270$$

In other words, crude Monte Carlo will converge towards the analytical probability when we increase our sample size parameter N .

For $N = 10^6$ we find:

```
[347]: N = 10**6 # number of simulations

def crude_monte_carlo(N, m, delta, gamma, sigma, x, loss_function):
    """ Crude Monte Carlo simulation of the probability P(L > x), note that all
    ↪ parameters should be of dimension m."""

    delta_S = np.random.multivariate_normal(np.zeros(m), sigma, size=N)
    L = loss_function(m, delta, delta_S, gamma)
    return np.mean(L > x), L
```

```
P_crude, L = crude_monte_carlo(N, m, delta, gamma, sigma, x, loss)
print(f"Crude Monte Carlo estimate of P(L > {x}): {P_crude:.6f}")
print(f"Difference to analytical: {abs(P_crude - analytical_prop):.6f}")
```

Crude Monte Carlo estimate of $P(L > 10.0)$: 0.010188

Difference to analytical: 0.000082

Which is already a quite small deviation, however for smaller N we find:

```
[348]: N = 100
P_crude_small, L_small = crude_monte_carlo(N, m, delta, gamma, sigma, x, loss)
print(
    f"Crude Monte Carlo estimate of P(L > {x}) with N={N}: {P_crude_small:.6f}")
print(f"Difference to analytical: {abs(P_crude_small - analytical_prop):.6f}")
```

Crude Monte Carlo estimate of $P(L > 10.0)$ with $N=100$: 0.000000

Difference to analytical: 0.010270

By the above, it is clear that our crude monte carlo estimate very much depends on the value of N .

Exercise b) Use control variables method to reduce the variance of the estimator from part (a). Discuss several possibilities and argue for your final choice. Would you recommend to use these methods given potential longer compute time?

In our Monte Carlo simulation, we want to determine $\mathbb{P}(L > x) = \mathbb{E}[\mathbf{1}_{L>x}] =: \ell$. We know that this can be done naively by the crude Monte Carlo utilizing the Law of Large numbers. However, if we want a high precision of the estimate, we might need a very high sample size N . Another way to determine the quality of our estimates, compared to the absolute difference in numerical value above, is the mean-squared error. For a given set of samples $Y_1, \dots, Y_n$ from the crude Monte Carlo we have:

$$\mathbb{E} \left[\left(\frac{(Y_1 + \dots + Y_n)}{N} - \mathbb{E}[Y_1] \right)^2 \right] = \mathbb{V} \left[\frac{(Y_1 + \dots + Y_n)}{N} \right] = \frac{1}{N} \mathbb{V}[Y_1].$$

The control variables method is one way to guarantee a variance reduction and thus increase the quality of our estimate of ℓ , at least by the above measure of quality. The main idea is that apart from sampling variables of the form $Y = \mathbf{1}_{L < x}$ as in the crude Monte Carlo, we also calculate Z , where $\mathbb{E}[Z]$ is known, and where Y and Z are positively correlated. We then use $\hat{Y} = Y - (Z - \mathbb{E}[Z])$ as our new variable for the Law of large numbers, indeed:

$$\mathbb{E}[\hat{Y}] = \mathbb{E}[Y] - 0 = \ell$$

In particular, we have shown in the lectures, that for the class of estimators given by

$$\hat{\ell} := \frac{1}{N} \sum_{i \leq N} [Y - \beta(Z_i - \ell_Z)]$$

there exists an optimal β given by

$$\beta_{\text{opt}} = \frac{\text{Cov}[Y, Z]}{\mathbb{V}(Z)}$$

and the minimum value of the variance is

$$\frac{1}{N} \mathbb{V}(Y) (1 - \text{Corr}[Y, Z]^2)$$

By the above, the a variance reduction is then guarenteed since Y and Z was assumed to be positively correlated.

To implement this method we will use the proposed estimator Q stated in the project description, and consider $\mathbf{1}_{Q > (x/2)}$. In particular, for the case at hand, we have $\mathbb{E}[\mathbf{1}_{Q > y}] = \mathbb{P}[\delta \Delta S \geq y] = 1 - \phi(y/\delta)$, since $\Delta S \sim N(0, 1)$.

```
[349]: def Q_estimator(Z, y, delta, m):
        if m == 1:
            Q = delta*Z
            mean = 1.0-norm.cdf(y/delta)
        else:
            Q = Z@delta
            mean = np.mean(Q > y) # approximate mean
        return Q > y, mean
```

We can then implement Control variance for this estimator

```
[350]: N = 10**6

def control_variance_monte_carlo(N, m, delta, x, control_variance_estimator, y):
    """ Control Variance monte carlo of P(L>x), note that all parameters shold
    ↪be of dimension m."""

    Z = np.random.multivariate_normal(np.zeros(m), sigma, N)
    Y = loss(m, delta, Z, gamma) > x
    H, l_H = control_variance_estimator(Z, y, delta, m)

    model = LinearRegression()
    beta = model.fit(H.reshape(-1, 1), Y).coef_[0]

    Y_hat = Y - beta*(H - l_H)

    return np.mean(Y_hat), Y, Y_hat
```

Using the function above one could try the control variate $\mathbf{1}_{Q > x/2}$ to get the following variance reduction:

```
[351]: P_cv_linear, Y_linear, Y_hat_linear = control_variance_monte_carlo(
        N, m, delta, x, Q_estimator, x/2)
print(
    f"Control Variance using Q in Monte Carlo estimate of P(L > {x}): ↪
    ↪{P_cv_linear:.6f}")
print(f"Variance reduction: {np.var(Y_linear)/np.var(Y_hat_linear):.2f} times")
```

Control Variance using Q in Monte Carlo estimate of $P(L > 10.0)$: 0.010277
Variance reduction: 2.46 times

An obvious question is how much variance reduction we could obtain by varying the y parameter in the control variate $\mathbf{1}_{Q>y}$, this is the content of the code below:

```
[352]: # find the best y to use in the control variate
Y = np.linspace(0, x, 100)
P_cv_list = []
variance_reduction_list = []
for y in Y:
    P_cv, Y_samples, Y_hat = control_variance_monte_carlo(
        N, m, delta, x, Q_estimator, y)
    P_cv_list.append(P_cv)
    variance_reduction_list.append(np.var(Y_samples)/np.var(Y_hat))
P_cv_list = np.array(P_cv_list)
variance_reduction_list = np.array(variance_reduction_list)
best_y = Y[np.argmax(variance_reduction_list)]
print(f"Best y: {best_y:.2f} with variance reduction of {np.
    ↪max(variance_reduction_list):.2f} times")
```

Best y: 4.65 with variance reduction of 55.86 times

In particular, our guess of $x/2 = 5$ is not that far from the best fitted y to the sample. We could also do a control variable on the quadratic part of L , for this estimator we have:

$$M = (\Delta S)^\top \Gamma \Delta S, \quad \mathbb{E}[\mathbf{1}_{M>y}] = \mathbb{P}((\Delta S)^2 > y) = 1 - (\phi(\sqrt{y}) - \phi(-\sqrt{y})) = 1 - (\phi(\sqrt{y}) - (1 - \phi(\sqrt{y}))) = 2\phi(\sqrt{y})$$

```
[353]: def M_estimator(Z, y, delta, m):
    if m == 1:
        M = (Z * gamma) * Z
        mean = 2*(1.0-norm.cdf(np.sqrt(y)))
    else:
        M = np.sum((Z @ gamma) * Z, axis=1)
        mean = np.mean(M > y) # approximate mean
    return M > y, mean
```

Using again the guess of $y = x/2$ we get:

```
[354]: P_cv_quadratic, Y_quadratic, Y_hat_quadratic = control_variance_monte_carlo(
    N, m, delta, x, M_estimator, x/2)
print(
    f"Control Variance using M in Monte Carlo estimate of P(L > {x}):_
    ↪{P_cv_quadratic:.6f}")
print(
    f"Variance reduction: {np.var(Y_quadratic)/np.var(Y_hat_quadratic):.2f}_
    ↪times")
```

Control Variance using M in Monte Carlo estimate of $P(L > 10.0)$: 0.010439
Variance reduction: 1.68 times

And again we can check our guess by simply doing a scan over different y values:

```
[355]: Y = np.linspace(0, x, 100)
P_cv_list = []
variance_reduction_list = []
for y in Y:
    P_cv, Y_samples, Y_hat = control_variance_monte_carlo(
        N, m, delta, x, M_estimator, y)
    P_cv_list.append(P_cv)
    variance_reduction_list.append(np.var(Y_samples)/np.var(Y_hat))
P_cv_list = np.array(P_cv_list)
variance_reduction_list = np.array(variance_reduction_list)
best_y = Y[np.argmax(variance_reduction_list)]
print(f"Best y: {best_y:.2f} with variance reduction of {np.
    ↪max(variance_reduction_list):.2f} times")
```

Best y: 5.35 with variance reduction of 1.97 times

Which again shows that our guess of $x/2 = 5$ is not that far from the best fitted y to the sample. The final control variate method we can look at is when we use both control variables, for simplicity we will simply use again $y_1 = y_2 = x/2$:

```
[356]: def control_variance_both_estimators(N, m, delta, sigma, gamma, x, y1, y2):
    """ Control Variance monte carlo of  $P(L > x)$ , note that all parameters should
    ↪be of dimension m. """
    Z = np.random.multivariate_normal(np.zeros(m), sigma, N)
    Y = loss(m, delta, Z, gamma) > x
    H1, l_H1 = Q_estimator(Z, y1, delta, m)
    H2, l_H2 = M_estimator(Z, y2, delta, m)

    H = np.column_stack((H1, H2))

    model = LinearRegression(fit_intercept=False)
    H_centered = H - np.array([l_H1, l_H2])
    beta = model.fit(H_centered, Y).coef_
    if m == 1:
        beta = beta[0]
        Y_hat = Y - beta[0]*(H1 - l_H1) - beta[1]*(H2 - l_H2)
    else:
        Y_hat = Y - H_centered@beta

    return np.mean(Y_hat), Y, Y_hat

P_cv_both, Y, Y_hat_both = control_variance_both_estimators(
    N, m, delta, sigma, gamma, x, x/2.0, x/2.0)
print(
```

```

    f"Control Variance using Q and M in Monte Carlo estimate of P(L > {x}):_
↪{P_cv_both:.6f}")
print(f"Variance reduction: {np.var(Y)/np.var(Y_hat_both):.2f} times")

```

Control Variance using Q and M in Monte Carlo estimate of $P(L > 10.0)$: 0.010355
Variance reduction: 3.16 times

Exercise c) Design an importance-sampling method to reduce the variance of the estimator from part (a). Again, discuss several options and argue for your final choice. Again, discuss the trade-off between additional compute time and variance reduction.

In importance sampling we want to find another density, which will give more information from each simulated value. In other words we want to find a density g , where the event $L > 10$ is more likely.

Let's start by computing the points x , fulfilling $L > 10$:

$$\begin{aligned}
 L &= x^2 + 2x > 10 \\
 &\Leftrightarrow x^2 + 2x + 1 > 11 \\
 &\Leftrightarrow x + 1 > \sqrt{11}, \quad x + 1 < -\sqrt{11} \\
 &\Leftrightarrow x_1 > 2.3166, \quad x_2 < -4.3166
 \end{aligned}$$

This means, that

$$\mathbb{P}(L > 10) = \mathbb{P}((\Delta S)^2 + 2\Delta S > 10) = \mathbb{P}(\Delta S < -4.3166) + \mathbb{P}(\Delta S > 2.3166)$$

Now it is worth noticing that since $\Delta S \sim N(0, 1)$, then

$$\mathbb{P}(\Delta S > 2.3166) = 1 - \Phi(2.3166) = 0.01022$$

$$\mathbb{P}(\Delta S < -4.3166) = \Phi(-4.3166) = 0.0000085$$

Which means that almost all of the contribution to $\mathbb{P}(L > 10)$ comes from $\Delta S > 2.3166$, i.e. the upper tail. Therefore, it wouldn't make much sense to do Importance Sampling (exponential tilting), where we set $x = -4.3166$. Now it is left to either do Importance Sampling (exponential tilting) with respect to $x = 2.3166$ or a mixture of the two. But, since the contribution of $(\Delta S < -4.3166)$ is that minor, a mixture would probably only result in more wasted samples.

Thus, we will only do importance sampling where we shift the distribution of ΔS to collect the effect of $\Delta S > 2.3166$.

This we will do by exponential tilting.

Therefore, let $x = \sqrt{11} - 1$ and we compute the optimal θ as the one fulfilling

$$\kappa'(\theta) = x$$

where $\kappa'(\theta)$ is given as the following, as $\Delta S \sim N(0, 1)$

$$\kappa'(\theta) = \mu + \sigma^2 \cdot \theta = \theta = \mathbb{E}_\theta[\Delta S]$$

Solving for θ gives

$$\theta = x$$

In the following we will use this $\kappa(\theta)$ (again as $\Delta S \sim N(0, 1)$):

$$\kappa(\theta) = \mu \cdot \theta + 0.5 \cdot \sigma^2 \cdot \theta^2 = 0 \cdot x + 0.5 \cdot 1^2 \cdot x^2 = 0.5\theta^2$$

And weights:

$$e^{-\theta \cdot \Delta S + \kappa(\theta)}$$

```
[357]: N = 10**6
mu = np.array([[0]])
x_tilt = np.sqrt(11) - 1

def IS_simulation(N, mu, delta, gamma, sigma, x):
    # Actually theta = x, but to be able to use for all values of mu and sigma
    # we write it like this
    theta = (x - mu) / (sigma**2)

    mu_tilted = mu + theta * (sigma**2)
    delta_S_tilted = np.random.normal(mu_tilted[0], sigma[0], N)

    L_tilted = delta_S_tilted**2 * gamma + delta * delta_S_tilted
    Z = (L_tilted > 10).astype(float)

    kappa = 0.5 * (theta**2)
    weights = np.exp(- theta * delta_S_tilted + kappa)

    weighted_Z = Z * weights
    running_estimate = np.cumsum(weighted_Z) / np.arange(1, N + 1)

    estimator = running_estimate[-1]
    se = np.std(weighted_Z, ddof=1) / np.sqrt(N)

    return estimator, se, running_estimate
```

```
[358]: # Exponential tilting with x=2.3166
p_est_1, se_1, running_estimate_1 = IS_simulation(N, mu, delta, gamma, sigma,
    x_tilt)
print(f"IS estimate for shift with x={x_tilt:.6f}: p_hat = {p_est_1:.6f},
    SE_hat = {se_1:.6f}")
```

IS estimate for shift with x=2.316625: p_hat = 0.010252, SE_hat = 0.000017

To emphasize that importance sampling where we shift ΔS to $N(-4.3166, 1)$ is not as effective, we consider the following:

```
[359]: x_tilt_2 = -np.sqrt(11) - 1
p_est_2, se_2, running_estimate_2 = IS_simulation(N, mu, delta, gamma, sigma,
    x_tilt_2)
```

```
print(f"IS estimate for shift with x={x_tilt_2:.6f}: p_hat = {p_est_2:.6f},  
      SE_hat = {se_2:.6f}")
```

IS estimate for shift with x=-4.316625: p_hat = 0.000008, SE_hat = 0.000000

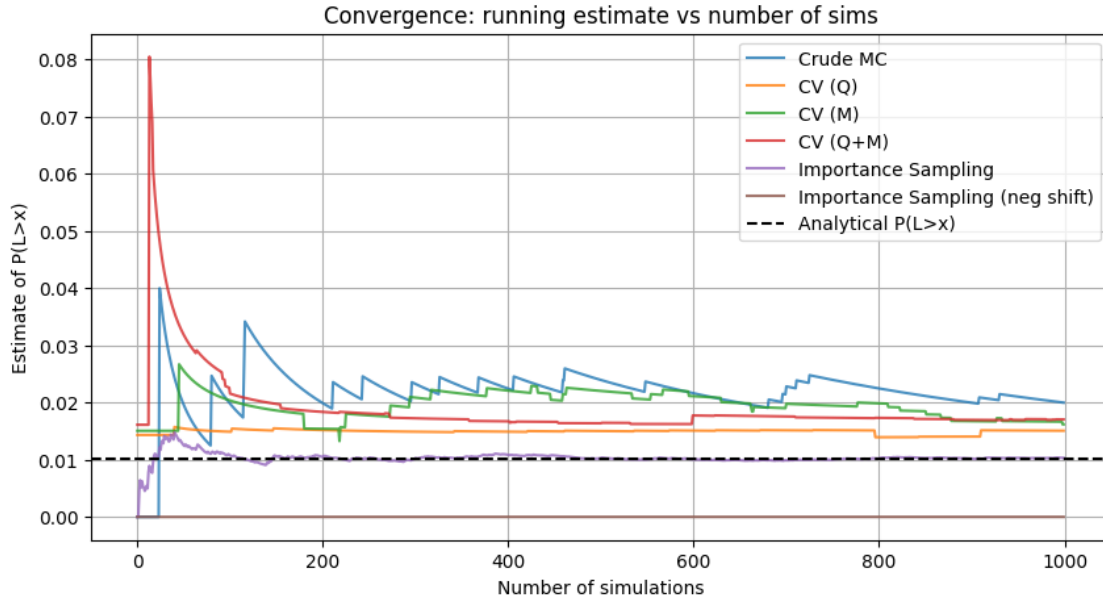
Where it can clearly be seen, that we do not get an exceedance probability close to the theoretical exceedance probability.

Exercise d) Create and discuss visualizations for comparing (a), (b) and (c).

```
[362]: # --- Plot 1: Convergence of running estimator
def running_mean(arr):
    return np.cumsum(arr) / np.arange(1, len(arr)+1)

nsim_to_use = 1000

plt.figure(figsize=(10, 5))
plt.plot(running_mean((L > x).astype(float)[
    :nsim_to_use]), label='Crude MC', alpha=0.8)
plt.plot(running_mean(Y_hat_linear[:nsim_to_use]), label='CV (Q)', alpha=0.8)
plt.plot(running_mean(
    Y_hat_quadratic[:nsim_to_use]), label='CV (M)', alpha=0.8)
plt.plot(running_mean(Y_hat_both[:nsim_to_use]), label='CV (Q+M)', alpha=0.8)
plt.plot(running_estimate_1[:nsim_to_use], label='Importance Sampling', alpha=0.8)
plt.plot(running_estimate_2[:nsim_to_use], label='Importance Sampling (neg_
shift)', alpha=0.8)
plt.axhline(analytical_prop, color='black',
    linestyle='--', label='Analytical P(L>x)')
plt.xlabel('Number of simulations')
plt.ylabel('Estimate of P(L>x)')
plt.title('Convergence: running estimate vs number of sims')
plt.legend()
plt.grid(True)
plt.show()
```



Exercise e) Show that for the specific choices of m, δ, Γ and Σ_S , the probability $\mathbb{P}(L > x)$ has a closed-form expression only involving the cdf of the standard normal distribution. Is this still true when choosing another distribution? For the variance-reduction methods in (b), (c), how easy do they generalize when L is replaced by another relevant distribution?

Question 1) Now, assume that $m = 2, \delta = (2, 2)^\top, \Gamma = Id_2, \Sigma_S = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix}$, and $x = 15$. Repeat parts (a) and (b) for these new parameters.

We have some new parameters and thus, the analytical means of Q and M become more difficult to determine. Therefore, we will simply use the sample means as the *analytical* means $\bar{Q}$ and $\bar{M}$. By the same argument, the analytical probability is also hard to determine.

```
[361]: m = 2
delta = np.array([2.0, 2.0])
gamma = np.identity(2)
sigma = np.array([[1.0, 0.5], [0.5, 1.0]])
x = 15.0
N = 10**6

# Crude Monte Carlo
P_crude, L = crude_monte_carlo(N, m, delta, gamma, sigma, x, loss)
```

```

# Control Variance using Q
P_cv_linear, Y_linear, Y_hat_linear = control_variance_monte_carlo(
    N, m, delta, x, Q_estimator, x/2.0)

# Control Variance using M
P_cv_quadratic, Y_quadratic, Y_hat_quadratic = control_variance_monte_carlo(
    N, m, delta, x, M_estimator, x/2.0)

# Control Variance using both Q and M
P_cv_both, Y, Y_hat_both = control_variance_both_estimators(
    N, m, delta, sigma, gamma, x, x/2.0, x/2.0)
print(f"Crude Monte Carlo estimate of P(L > {x}): {P_crude:.6f}")
print(
    f"Control Variance using Q in Monte Carlo estimate of P(L > {x}): {P_cv:.6f}")
print(f"Variance reduction: {np.var(Y_linear)/np.var(Y_hat_linear):.2f} times")
print(
    f"Control Variance using M in Monte Carlo estimate of P(L > {x}): {P_cv_quadratic:.6f}")
print(
    f"Variance reduction: {np.var(Y_quadratic)/np.var(Y_hat_quadratic):.2f} times")
print(
    f"Control Variance using Q and M in Monte Carlo estimate of P(L > {x}): {P_cv_both:.6f}")
print(f"Variance reduction: {np.var(Y)/np.var(Y_hat_both):.2f} times")

# create a plot of the convergence of the probability estimates
plt.figure(figsize=(10, 5))
plt.plot(running_mean((L > x).astype(float)[
    :nsim_to_use]), label='Crude MC', alpha=0.8)
plt.plot(running_mean(Y_hat_linear[:nsim_to_use]), label='CV (Q)', alpha=0.8)
plt.plot(running_mean(
    Y_hat_quadratic[:nsim_to_use]), label='CV (M)', alpha=0.8)
plt.plot(running_mean(Y_hat_both[:nsim_to_use]), label='CV (Q+M)', alpha=0.8)
plt.xlabel('Number of simulations')
plt.ylabel('Estimate of P(L>x)')
plt.title('Convergence: running estimate vs number of sims')
plt.legend()
plt.grid(True)
plt.show()

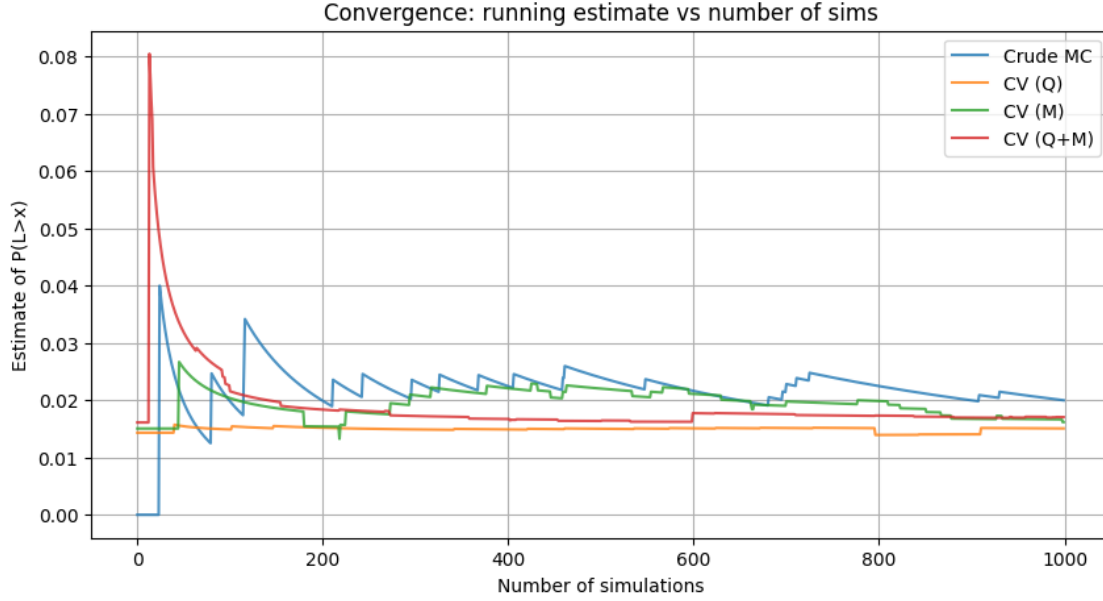
```

```

Crude Monte Carlo estimate of P(L > 15.0): 0.015678
Control Variance using Q in Monte Carlo estimate of P(L > 15.0): 0.010165
Variance reduction: 7.62 times
Control Variance using M in Monte Carlo estimate of P(L > 15.0): 0.015500
Variance reduction: 1.80 times

```

Control Variance using Q and M in Monte Carlo estimate of $P(L > 15.0)$: 0.015487
Variance reduction: 8.69 times



Exercise f) Now, use the Cholesky decomposition to find a matrix $\tilde{C}$ such that $\Delta S = \tilde{C}Z$ where $Z \sim N(0, I_2)$. Next, find an orthogonal matrix U and a diagonal matrix $\Lambda = \text{diag}(\lambda_1, \lambda_2)$ such that $\tilde{C}^\top \tilde{C} = U\Lambda U^\top$. Show that the loss L can be expressed as

$$L = b_1 Z_1 + b_2 Z_2 + \lambda_1 Z_1^2 + \lambda_2 Z_2^2$$

where $b = C^\top \delta$ with $C = \tilde{C}U$.

Cholesky decomposition

We want $\Delta S = \tilde{C}Z$, where ΔS follows a multivariate normal distribution. By properties of the multivariate normal distribution this is satisfied when $\Sigma_S = \tilde{C}I_2\tilde{C}^\top = \tilde{C}\tilde{C}^\top$. This shows that the desired matrix $\tilde{C}$ can be obtained as the cholesky decomposition of Σ_S , since Σ_S has full rank. This decomposition can be found using a recursive algorithm found [here](#). The following shows our calculations when using the algorithm to decompose Σ_S :

$$(i = 1) \quad A^{(1)} := \Sigma_S = \begin{pmatrix} 1 & \frac{1}{2} \\ \frac{1}{2} & 1 \end{pmatrix} \implies L_1 = \begin{pmatrix} 1 & 0 \\ \frac{1}{2} & 1 \end{pmatrix}$$

$$(i = 2) \quad A^{(2)} = \begin{pmatrix} 1 & 0 \\ 0 & \frac{3}{4} \end{pmatrix} \implies L_2 = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{\frac{3}{4}} \end{pmatrix}$$

$$(i = 3) \quad A^{(3)} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

At which point the algorithm terminates, $\tilde{C}$ is then obtained as follows:

$$L_1 L_2 = \begin{pmatrix} 1 & 0 \\ \frac{1}{2} & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{\frac{3}{4}} \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ \frac{1}{2} & \sqrt{\frac{3}{4}} \end{pmatrix} := \tilde{C}$$

As a sanity check we indeed have that:

$$\tilde{C} \tilde{C}^\top = \begin{pmatrix} 1 & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{4} + \frac{3}{4} \end{pmatrix} = \Sigma_s$$

Eigen decomposition of $\tilde{C}$

Note that

$$\tilde{C}^\top \tilde{C} = \begin{pmatrix} \frac{5}{4} & \frac{\sqrt{3}}{4} \\ \frac{\sqrt{3}}{4} & \frac{3}{4} \end{pmatrix}$$

is a real symmetric 2x2 matrix. This implies that the Eigen decomposition $\Sigma_s = U \Lambda U^\top$, with U orthogonal and Λ a diagonal matrix, has a closed form expression. i.e. for

$$A = \begin{pmatrix} a & b \\ b & d \end{pmatrix}$$

the eigen values are given by

$$\lambda_1 = \frac{a+d}{2} - \Delta, \quad \lambda_2 = \frac{a+d}{2} + \Delta$$

with $\Delta := \sqrt{\left(\frac{a-d}{2}\right)^2 + b^2}$. For our values:

$$\Delta = \frac{1}{2} \implies \lambda_1 = \frac{1}{2}, \quad \lambda_2 = \frac{3}{2}$$

For an eigenvalue λ a non-trivial eigenvector can be found by

$$v_\lambda = \begin{pmatrix} -b \\ a - \lambda \end{pmatrix},$$

Which we can normalize to unit length:

$$u_\lambda = \frac{v_\lambda}{\|v_\lambda\|}$$

For our two eigenvalues we then get two orthonormal eigenvectors $u_1 := u_{\lambda_1}$, $u_2 := u_{\lambda_2}$. The matrix U given by $U = [u_1 u_2]$ is then orthogonal, we have:

$$v_{\lambda_1} = \begin{pmatrix} -\frac{\sqrt{3}}{4} \\ \frac{3}{4} \end{pmatrix}, \quad v_{\lambda_2} = \begin{pmatrix} -\frac{\sqrt{3}}{4} \\ -\frac{1}{4} \end{pmatrix} \implies u_{\lambda_1} = \begin{pmatrix} -\frac{1}{2} \\ \frac{\sqrt{3}}{2} \end{pmatrix} \quad u_{\lambda_2} = \begin{pmatrix} -\frac{\sqrt{3}}{2} \\ -\frac{1}{2} \end{pmatrix} \implies U = \begin{pmatrix} -\frac{1}{2} & -\frac{\sqrt{3}}{2} \\ \frac{\sqrt{3}}{2} & -\frac{1}{2} \end{pmatrix}$$

Thus, we have arrived at our eigen decomposition:

$$U = \begin{pmatrix} -\frac{1}{2} & -\frac{\sqrt{3}}{2} \\ \frac{\sqrt{3}}{2} & -\frac{1}{2} \end{pmatrix}, \quad \Lambda = \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & \frac{3}{2} \end{pmatrix}$$

Loss identity

The loss is given as:

$$L = \delta^\top \Delta S + (\Delta S)^\top \Gamma \Delta S = L = \delta^\top \tilde{C}Z + (\tilde{C}Z)^\top \Gamma \tilde{C}Z$$

With $\Gamma = I_2$ we get:

$$L = \delta^\top \tilde{C}Z + (\tilde{C}Z)^\top \tilde{C}Z = \delta^\top \tilde{C}Z + Z^\top (\tilde{C}^\top \tilde{C})Z = \delta^\top \tilde{C}Z + Z^\top (U\Lambda U^\top)Z$$

with $C = \tilde{C}U$ and since U is orthogonal we have:

$$L = \delta^\top \tilde{C}Z + Z^\top (U\Lambda U^\top)Z = \delta^\top (\tilde{C}U)U^\top Z + Z^\top (U\Lambda U^\top)Z = \delta^\top CU^\top Z + Z^\top (U\Lambda U^\top)Z$$

with $b = \delta^\top C$ this becomes:

$$L = b^\top U^\top Z + Z^\top (U\Lambda U^\top)Z = b^\top (U^\top Z) + (Z^\top U)\Lambda(U^\top Z)$$

Now since $Z \sim N(\mathbf{0}, I_2)$ and U is orthogonal $U^\top Z = Z$ and we have:

$$L = b^\top Z + Z\Lambda Z = b_1 Z_1 + b_2 Z_2 + \lambda_1 Z_1 + \lambda_2 Z_2$$

Which is what we wanted to show.

Exercise g) Similarly as in part (c), use $\prod_{i \leq 2} \exp(\theta_i Z_i - \psi^*(\theta_i))$ as an importance-sampling density, where ψ^* is the cumulant generating function of a standard normal distribution. Experiment with different parameters θ_i .

Exercise h) For $(\lambda_1 \vee \lambda_2)\theta < 1/2$ show that

$$\psi_L(\theta) := \log \mathbb{E}[\exp(\theta L)] = \frac{1}{2} \sum_{j \leq 2} \left(\frac{\theta^2 b_j^2}{1 - 2\theta \lambda_j} - \log(1 - 2\theta \lambda_j) \right).$$

You may use without proof that

$$\mathbb{E} \left[\exp \left(\theta (Z_1 + c)^2 \right) \right] = (1 - 2\theta)^{-1/2} \exp(\theta c^2 / (1 - 2\theta))$$

holds for all $c \in \mathbb{R}$ and $\theta < 1/2$.

Exercise i) Use part (h) to develop an importance-sampling algorithm for the estimation of $\mathbb{P}(L > 15)$. Develop a heuristic for choosing θ .

Exercise j) Describe broadly how stratified sampling could be used for simulating L when the random variable Q is used for stratification. For this question, a description of your approach is sufficient; you do not need to implement a simulation program.